

PROMPT IA 09

IAM - Keycloak + OIDC/OAuth2 + MFA/Passkeys + RBAC

Customer + Admin + Kong + Istio + OPA + Backend Authoritative

Mission : implementer l'identite, l'authentification, les sessions, les roles et les autorisations fines de la plateforme e-commerce, avec Keycloak comme autorite IAM unique.

Security-first - least privilege - short-lived tokens - no Redis sessions - backend authoritative

1. Role de l'IA

Tu es Principal IAM/Security Engineer. Tu dois implementer une couche d'identite exploitable par le Customer Storefront, l'Admin Backoffice, Kong, Istio Ambient, OPA et les microservices Go. Le resultat doit etre reproductible, teste, auditable et deployable via GitOps.

Keycloak reste l'unique autorite pour sessions, access tokens, refresh tokens, rotation et revocation. Redis ne gere aucune session ou token d'authentification.

2. Architecture cible

```
Customer / Admin
|
v
Keycloak
OIDC / OAuth2
MFA / Passkey
|
v
Kong
JWT/OIDC validation
|
v
Istio Ambient
identity + mTLS
|
v
Backend Services
|
+--> OPA for Platform API authorization
+--> domain authorization checks
```

3. Populations et clients

Population	Client / usage	Regle
Customer	Customer Frontend Next.js	OIDC public client / PKCE, sessions utilisateur.
Admin	Admin Backoffice Next.js	OIDC, MFA/Passkey renforce, roles privileges.
Platform users	Backstage / Platform API	OIDC avec policies OPA.
Machine workloads	SPIFFE/SPIRE	Ne pas utiliser des comptes humains Keycloak comme identite machine.

4. Roles et permissions

Implementer les roles de base deja valides et leurs permissions fines. Les roles sont des agregats de permissions, pas une logique metier codee en dur dans le frontend.

```
Roles:
- super_admin
- catalog_manager
- inventory_manager
- order_manager
- finance_manager
- support_agent
- customer

Permissions exemples:
- product:write
- inventory:adjust
- order:refund
- order:read
```

- payment:read
- payment:refund
- review:moderate

Construire une matrice role -> permissions versionnee et testee. Le backend doit verifier la permission requise a chaque operation sensible.

5. Authentification Customer

Utiliser Authorization Code + PKCE. Access tokens courts, refresh tokens rotatifs, scopes minimaux. Supporter MFA optionnel/conditionnel selon politique et Passkeys/WebAuthn lorsque le parcours produit l'autorise.

Le Customer Frontend ne stocke jamais des tokens sensibles dans localStorage si une alternative plus sure est disponible. Preferer des mecanismes de session web adaptes au modele Next.js et aux protections CSRF/XSS.

6. Authentification Admin

Pour l'Admin Backoffice, imposer MFA ou Passkey selon la politique de securite, avec step-up authentication pour les actions sensibles : remboursement important, modification IAM, regles Pricing/Tax/Fraud, acces production ou action Tier 0.

```
Admin login
-> Keycloak
-> MFA / Passkey
-> token short-lived
-> Kong
-> Backend
-> permission check
-> audit
```

7. Sessions, tokens et revocation

Keycloak gere les sessions. Configurer durees de vie, refresh token rotation, detection de reutilisation et revocation.

```
Access token
-> courte duree
-> validation iss/aud/exp
```

```
Refresh token
-> rotation
-> revocable
-> reuse detection
```

```
Session Keycloak
-> source de verite
```

Un compte desactive doit perdre l'accès rapidement. Documenter le comportement des access tokens deja emis et utiliser des TTL courts pour borner la fenetre de risque.

8. Validation Kong

Configurer Kong pour verifier les tokens OIDC/JWT en entree. Verifier au minimum issuer, audience, expiration, signature et scopes requis selon endpoint. Ne pas deleguer toute l'autorisation metier a Kong.

```
Internet
-> Kong
- token valid ?
- issuer/audience ?
- scope technique ?
-> Backend
- permission metier ?
- invariant metier ?
```

9. Istio Ambient et identite interne

Conserver SPIFFE/SPIRE pour les workloads et Istio Ambient pour mTLS/service identity. Ne pas propager un token humain la ou une identite workload suffit.

Lorsque le contexte utilisateur doit etre propage, definir une strategie explicite et limitee, avec correlation/audit, sans transformer tous les appels internes en simple forwarding de bearer tokens.

10. OPA et Platform API

OPA est le Policy Decision Point de la Platform API. Keycloak fournit l'identite humaine ; OPA decide ALLOW / APPROVAL / DENY selon role, owner, environnement, Tier, quota et intent.

```
Backstage / CLI
-> Keycloak
-> Platform API
-> OPA
+> ALLOW
+> APPROVAL
+> DENY
```

11. Backend authoritative

Le frontend peut masquer/afficher des boutons, mais aucune permission critique ne depend de l'UI. Toute decision d'autorisation finale est revalidee cote backend.

Admin UI: bouton Refund masque si role absent

```
Mais:
POST /refund
-> backend verifie permission order:refund
-> backend verifie regles metier
-> backend execute ou refuse
```

12. Security events et audit

Produire des evenements d'audit pour login admin, echec MFA, revocation, changement de role, elevation, modification client OIDC et action IAM sensible. Envoyer les signaux securite pertinents vers Splunk sans journaliser les tokens complets.

```
audit fields:
actor_id
action
resource
result
request_id
correlation_id
trace_id
timestamp
source_ip / context when justified
```

13. Secrets et configuration

Les secrets clients confidentiels, credentials techniques et materiel cryptographique proviennent d'OpenBao. Les configurations non sensibles Keycloak sont versionnees comme code autant que possible.

Ne jamais placer un client secret permanent directement dans un repository Git ou un ConfigMap.

14. Configuration-as-Code

Versionner realms, clients, roles, scopes, protocol mappers et policies quand le mode de deploiement choisi le permet. Les changements IAM de production passent par PR, tests et approbation selon criticite.

```
git/iam/
realms/
clients/
roles/
scopes/
mappings/
```

15. Tests obligatoires

Scenario	Attendu
Customer login PKCE	Login OK, token valide, scopes minimaux.
Admin sans MFA	Acces refuse si politique MFA obligatoire.
Role support_agent	Acces lecture support, pas de refund finance si non autorise.
Token expire	401 propre, aucun fallback insecure.
Audience invalide	Rejet Kong/backend.
Refresh token replay	Detection/revocation selon configuration.
User desactive	Nouvel acces refuse, sessions/revocation appliquees.
Permission frontend contournee	Backend refuse quand meme l'operation.
Step-up auth	Action sensible exige authentication renforcee.
Audit	Evenement IAM/security correctement produit.

16. Observabilite IAM

```
login_success_total
login_failure_total
mfa_failure_total
refresh_token_reuse_total
active_sessions
revocation_total
unauthorized_requests_total
forbidden_requests_total
keycloak_request_latency
```

Exporter metrics vers Prometheus/Thanos et construire dashboards Grafana/Backstage. Alertes sur anomalies de login, erreurs massives OIDC et indisponibilite Keycloak.

17. Haute disponibilite et PRA

Keycloak doit etre hautement disponible par site, avec PostgreSQL gere par CloudNativePG selon la topologie retenue. Tester failover, perte d'instance et restauration.

Le design multi-site doit rester coherent avec deux sites autonomes. Aucun mode ne doit introduire une dependance WAN obligatoire pour authentifier les utilisateurs locaux.

18. Ordre d'implementation

1. Realm + clients Customer/Admin
2. Roles + permissions + scopes
3. Customer OIDC PKCE
4. Admin MFA/Passkey
5. Kong validation OIDC/JWT
6. Backend permission middleware/use cases
7. Step-up auth actions sensibles
8. OPA integration Platform API
9. Audit/security events
10. Metrics/dashboards/alerts
11. HA/failover tests
12. GitOps/IaC + documentation

19. Definition of Done

- Customer peut se connecter via OIDC Authorization Code + PKCE.
- Admin doit utiliser la politique MFA/Passkey definie.
- Kong rejette les tokens invalides avant le backend.
- Le backend revalide toutes les permissions metier sensibles.
- Les roles et permissions de base sont couverts par des tests.
- La revocation et la rotation des refresh tokens sont testees.
- Aucune session applicative n'est stockee dans Redis.
- Les secrets IAM ne sont pas stockes en clair dans Git/Kubernetes.
- Les actions IAM sensibles sont auditees.
- La configuration est reproductible et deployable via GitOps/IaC.

Le livrable n'est termine que si un test de contournement frontend prouve qu'une action interdite est toujours refusee par le backend. L'UI n'est jamais une barriere de securite.

20. Regles de travail pour l'IA

Implemente par increments courts. Apres chaque increment : build, tests, correction, factorisation. Evite les duplications de verification JWT/permissions. Centralise les primitives de securite communes sans creer un framework interne excessif.

Ne modifie pas les regles de role/permission sans les documenter. Si une exigence securite est ambiguë, choisis le comportement le plus restrictif compatible avec le parcours et expose l'hypothese dans la documentation/ADR.